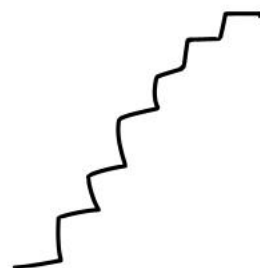


PMI

$$\sum_{i=1}^n i = \frac{n(n+1)}{2}$$

$$1 \leq k \leq n$$



① $\sum 1 = 1$

② Assume $\sum k = \frac{k(k+1)}{2}$

③ Prove $\sum_{k+1} = \frac{(k+1)(k+2)}{2}$

$$\sum_{k+1} = \sum k + k+1$$

$$= \frac{k(k+1)}{2} + k+1$$

$$= \frac{k(k+1) + 2(k+1)}{2}$$

$$= \frac{(k+1)(k+2)}{2}$$

def fact(n):

if $n \leq 1$:

return n

smallOutput = fact(n-1)

return $n \times \text{smallOutput}$ Induction
step

Induction Hypothesis

PMI

① $\rightarrow \text{fact} \rightarrow 1 \Rightarrow 1$

② $\rightarrow$ Assume $(n-1)!$

③ Find $n!$

$$n \times (n-1)!$$

0	1	2	3	4	5
10	20	40	30	25	20

elem [20]

Find first
occurrence of elem:

o/p: 1

0	1	2	3	4	5
10	20	40	30	25	20

elem [200]

[-1]

-1

←

← -1

0	1
10	20

elem [30]

i [0]

0	1
10	20

elem [30]

i [1]

0	1
10	20

elem [30]

i [2]

```

2 def firstOccurrence(nums, element, i):
3     if i == len(nums):
4         return -1
5     if nums[i] == element:
6         return i
7     return firstOccurrence(nums, element, i + 1)
8
9
10 nums = [10, 20, 40, 30]
11 print(firstOccurrence(nums, 200, 0))

```

0	1	2	3
10	20	40	30

Can't find
-1

fo()	elem [200]	i [4]
fo()	elem [200]	i [3]
fo()	elem [200]	i [2]
fo()	elem [200]	i [1]
fo()	elem [200]	i [0]
main()		

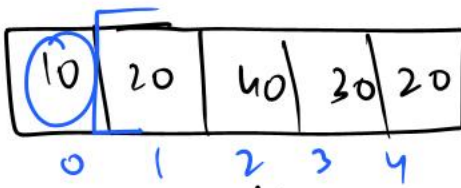
7 -1

7 -1

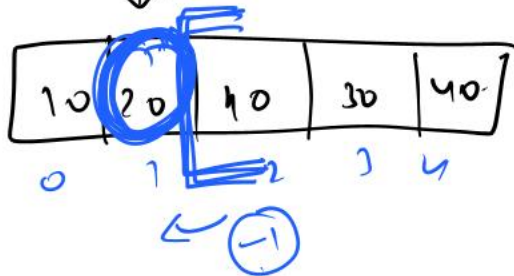
7 -1

7 -1

11 -1



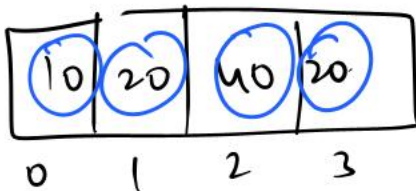
o/p: 4



```

9 def lastOccurrence(nums, element, i) :
10     if i == len(nums) :
11         return -1
12     if nums[i] != element :
13         return lastOccurrence(nums, element, i + 1)
14     else :
15         ans = lastOccurrence(nums, element, i + 1)
16         if ans != -1 :
17             return ans
18         else :
19             return i
20
21 nums = [10, 20, 40, 20]
22 # print(firstOccurrence(nums, 200, 0))
23 print(lastOccurrence(nums, 20, 0))

```



3

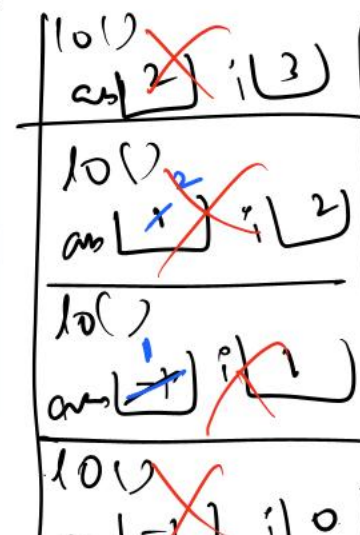


15
13
15
13
23

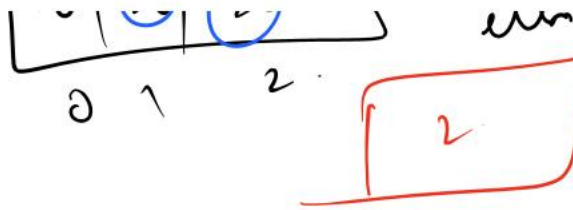
```

21 def lastOccurrence1(nums, element, i, ans) :
22     if i == len(nums) :
23         return ans
24     if nums[i] == element :
25         ans = i
26     return lastOccurrence1(nums, element, i + 1, ans)
27
28 nums = [10, 20, 20]
29 print(lastOccurrence1(nums, 20, 0, -1))
30 # print(firstOccurrence(nums, 200, 0))
31 # print(lastOccurrence(nums, 200, 0))

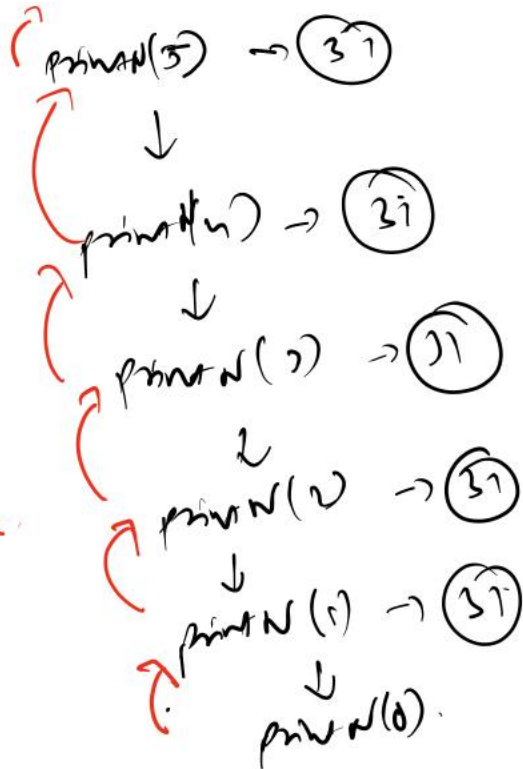
```



26
26
26



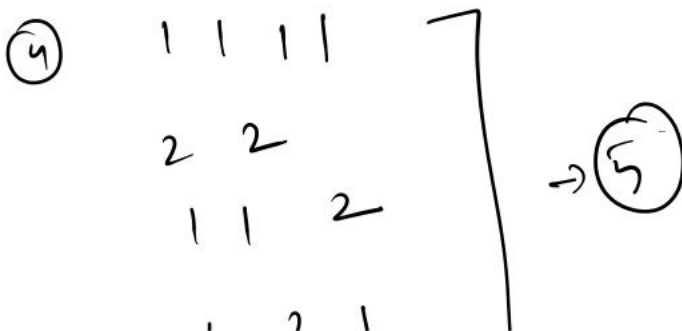
```
28 def printN(n) :
29     if n == 0 :
30         return
31     printN(n - 1)
32     print(n, end = " ")
33
34
35 printN(5)
```



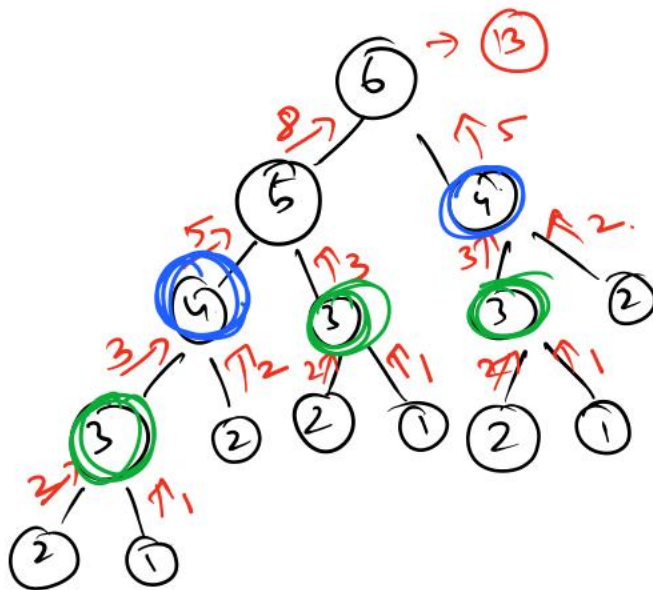
70. Climbing Stairs

Easy

You are climbing a staircase. It takes n steps to reach the top.



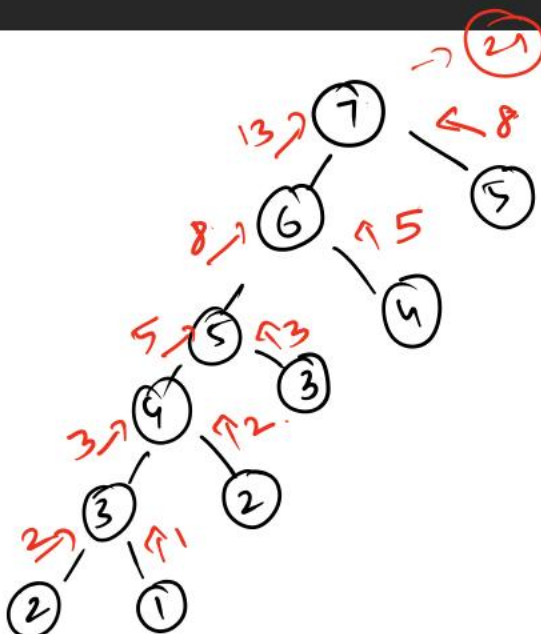
1 4 1
2 1 1



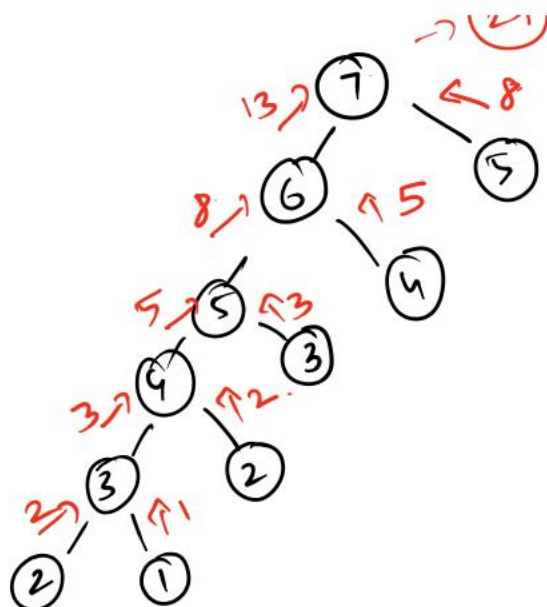
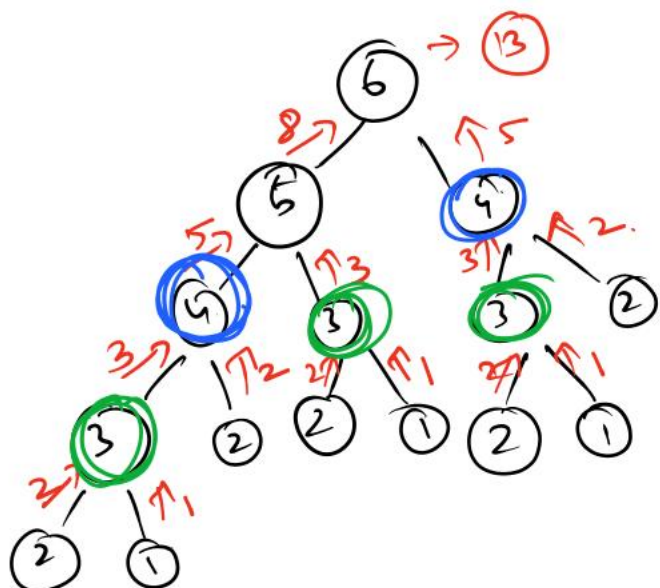
① → 1
② → $\begin{matrix} 1 & 1 \\ 2 \end{matrix} \rightarrow 2$

```
1 class Solution:
2     def climbStairs(self, n: int) -> int:
3         if n == 1 or n == 2 :
4             return n
5         ans1 = self.climbStairs(n - 1)
6         ans2 = self.climbStairs(n - 2)
7         return ans1 + ans2
```

⑤
 $\frac{2 \times 2 \times 2 \times 2 \times 2}{2^5}$
②⁵



3 : 3
4 : 5
5 : 8
6 : 13
map



```
class Solution:
```

```
def climbStairs(self, n: int) -> int:
```

```
def helper(n, map):
```

```
    if n == 1 or n == 2:
```

```
        return n
```

```
    ans1 = -1
```

```
    ans2 = -1
```

```
    if n - 1 in map:
```

```
        ans1 = map[n - 1]
```

```
    else:
```

```
        ans1 = helper(n - 1, map)
```

```
        map[n - 1] = ans1
```

```
    if n - 2 in map:
```

```
        ans2 = map[n - 2]
```

```
    else:
```

```
        ans2 = helper(n - 2, map)
```

```
        map[n - 2] = ans2
```

```
    return ans1 + ans2
```

```
map = {}
```

```
return helper(n, map)
```

→ RECURSIVE DP

checking if n-1 ans is already present

checking if n-2 ans is already present

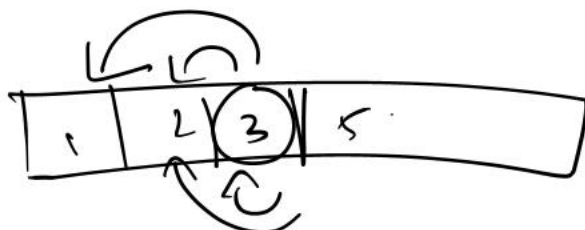
→ P. NO. (70)

T.C → $O(n)$

S.C → $O(n)$

dictionary size + call stack

⑥



1 → (1)

$n = 1$
 $n = 2 \rightarrow 2$

→ ITERATIVE DP.

class Solution:

```

def climbStairs(self, n: int) -> int:
    if n == 1 or n == 2 :
        return n
    dp = [1, 2]
    for i in range(2, n) :
        ans1 = dp[i - 1]
        ans2 = dp[i - 2]
        dp.append(ans1 + ans2)
    return dp[len(dp) - 1]

```

→ T.C → $O(n)$

→ S.C → $O(n)$

dp array
size

$n \rightarrow 10^5 \rightarrow$ call stack → accommodate $\rightarrow 10^4$ function

↓

no. of function $> 10^4$

Stack Overflow
Error

2D Matrices.

let $n \times m$

matrix = []

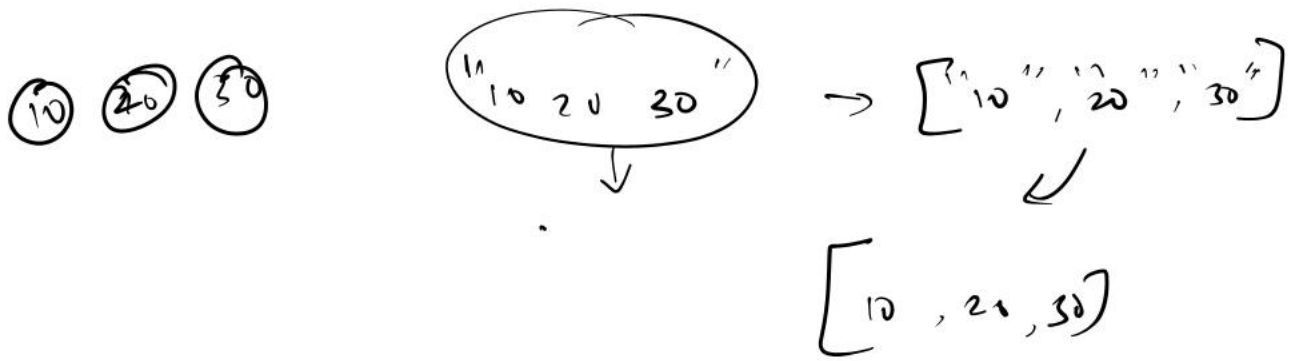
③

1 2 3

4 5 6 7

10

1	2	3	
4	5	6	7
10			



1672. Richest Customer Wealth

Solved

Easy Topics Companies Hint

You are given an $m \times n$ integer grid `accounts` where `accounts[i][j]` is the amount of money the i^{th} customer has in the j^{th} bank. Return the wealth that the richest customer has.

A customer's **wealth** is the amount of money they have in all their bank accounts. The richest customer is the customer that has the maximum wealth.

Example 1:

Input: `accounts = [[1,2,3],[3,2,1]]`
 Output: 6

```
class Solution:
    def maximumWealth(self, accounts: List[List[int]]) -> int:
        output = 0
        for i in range(len(accounts)):
            currentWealth = 0
            for j in range(len(accounts[i])):
                currentWealth += accounts[i][j]
            output = max(output, currentWealth)
        return output
```

```
class Solution:
    def maximumWealth(self, accounts: List[List[int]])
    -> int:
        output = 0
        for row in accounts:
            currentWealth = sum(row)
            output = max(currentWealth, output)
        return output
```

P.N.D. 1672
 T.C $\rightarrow O(m \times n)$
 S.C $\rightarrow O(1)$

999. Available Captures for Rook

Solved

Easy

Topics

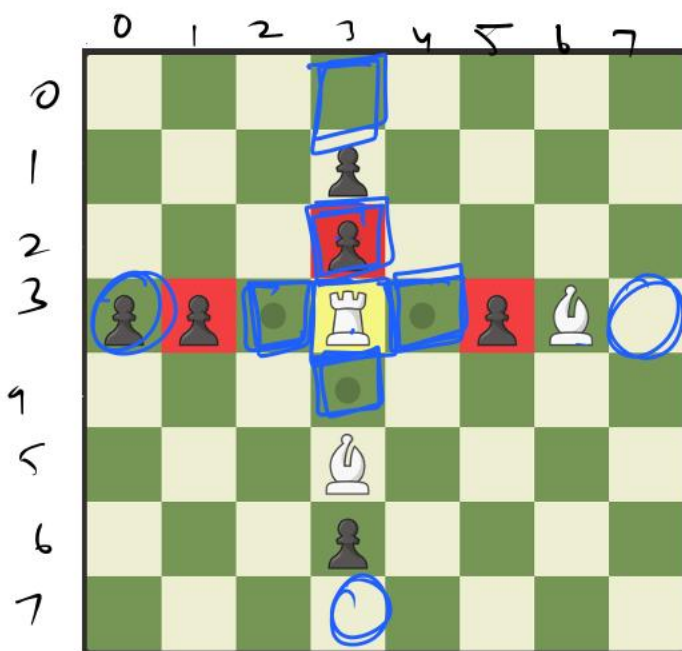
Companies

You are given an 8×8 matrix representing a chessboard. There is **exactly one** white rook represented by 'R', some number of white bishops 'B', and some number of black pawns 'p'. Empty squares are represented by '.'.

A rook can move any number of squares horizontally or vertically (up, down, left, right) until it reaches another piece or the edge of the board. A rook is **attacking** a pawn if it can move to the pawn's square in one move.

Note: A rook cannot move through other pieces, such as bishops or pawns. This means a rook cannot attack a pawn if there is another piece blocking the path.

Return the **number of pawns** the white rook is attacking.



$R \rightarrow [3,3]$ $i[3]$ $j[3]$

Direction	Common	Start	End
Up	$col(i)$	2	0
Right	$row(i)$	4	7
Down	$col(j)$	4	7
Left	$row(i)$	2	0

class Solution:

def numRookCaptures(self, board: List[List[str]]) -> int:

 pawns = 0

 # Find rook position

 for i in range(8):

 for j in range(8):

 if board[i][j] == 'R':

 # Up -> col is same, row -> i - 1 to 0

 for k in range(i - 1, -1, -1):

 if board[k][j] == 'B':

 break

 elif board[k][j] == 'p':

 pawns += 1

 break

 # Down -> col is same, row -> i + 1 to 7

 for k in range(i + 1, 8):

 if board[k][j] == 'B':

 # Right -> row is same, col -> j + 1 to 7

 for k in range(j + 1, 8):

 if board[i][k] == 'B':

 break

 elif board[i][k] == 'p':

 pawns += 1

 break

 # Left -> row is same, col -> j - 1 to 0

 for k in range(j - 1, -1, -1):

 if board[i][k] == 'B':

 break

 elif board[i][k] == 'p':

 pawns += 1

 break

 return pawns

```

        break
    elif board[k][j] == 'p':
        pawns += 1
        break

```

→ T.C → O(1) S.C → O(1)

T.C → O(1) S.C → O(1)

344. Reverse String

Easy Topics Companies Hint

Write a function that reverses a string. The input string is given as an array of characters `s`.

You must do this by modifying the input array in-place with O(1) extra memory.

Example 1:

Input: `s = ["h","e","l","l","o"]`

Output: `["o","l","l","e","h"]`

Example 2:

Input: `s = ["H","a","n","n","a","h"]`

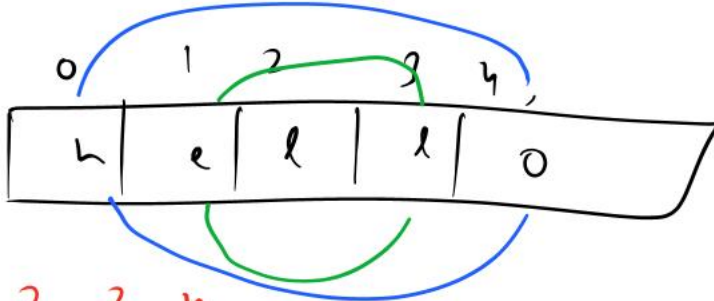
Output: `["h","a","n","n","a","H"]`

["h", "e", "l", "l", "o"]
↓

op →

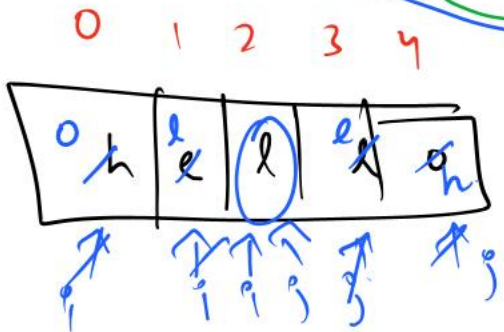
["o", "l", "l", "e", "h"]

['h', 'e', 'l', 'l', 'o']



0 → 4
4 → 0

1 → 3
3 → 1



i = 1, j = 3

j = 2, i = 4

→ i == j

2 → i = 2, j = 2

1 → i = 1, j = 3

i > j → stop



i < j → start swap



```
class Solution:
```

```
def reverseString(self, s: List[str]) -> None:
```

```
    """
```

```
    Do not return anything, modify s in-place instead.
```

```
    """
```

```
    i, j = 0, len(s) - 1
```

```
    while i < j:
```

```
        s[i], s[j] = s[j], s[i]
```

```
        i += 1
```

```
        j -= 1
```

→ P.No (344)

→ T.C → $O(n)$

→ S.C → $O(1)$

125. Valid Palindrome

Solved ✓

Easy

Topics

Companies

A phrase is a **palindrome** if, after converting all uppercase letters into lowercase letters and removing all non-alphanumeric characters, it reads the same forward and backward. Alphanumeric characters include letters and numbers.

Given a string `s`, return `true` if it is a **palindrome**, or `false` otherwise.

Example 1:

Input: `s = "A man, a plan, a canal: Panama"`

Output: `true`

Explanation: "amanaplanacanalpanama" is a palindrome.

"--?1?a--!"

↓

1a1

↓

(True)

"--?--a--b!--B--?--A--?!"
(Handwritten annotations: red 'x' marks over the first three characters, red 'i' under the 'a', blue 'j' under the 'A', and blue 'x' marks over the last three characters)

```
class Solution:
```

```
def isPalindrome(self, s: str) -> bool:
```

```
    i, j = 0, len(s) - 1
```

```
    while i < j:
```

```
        # Ignore the non alphanumeric characters in left
```

```
        while i < j and not s[i].isalnum():
```

```
            i += 1
```

```
        # Ignore the non alphanumeric characters in right
```

```
        while i < j and not s[j].isalnum():
```

```
            j -= 1
```

→ P.No (125)

→ T.C → $O(n)$

17

```

c1 = s[i].lower()
c2 = s[j].lower()
if c1 != c2 :
    return False
i += 1
j -= 1
return True

```

→ j, c → 0 1 2

242. Valid Anagram

Solved ✓

Easy Topics Companies

Given two strings `s` and `t`, return `true` if `t` is an `anagram` of `s`, and `false` otherwise.

Example 1:

Input: `s = "anagram", t = "nagaram"`

Output: `true`

Example 2:

Input: `s = "rat", t = "car"`

Output: `false`

s → "anagram"

t → "nagaram"

↓
true

"anagram"

"nagaram"

a: x23^{or}1
 n: x0
 g: x0
 r: x0
 m: x0

"hello"

"olleh" (e circled in red)

h : x⁰
 e : x⁶ → false
 l : x²
 o : x⁰

```
class Solution:
    def isAnagram(self, s: str, t: str) -> bool:
        if len(s) != len(t) :
            return False
        map = {}
        # Exploring all characters in String s and noting its frequency
        for c in s :
            if c in map :
                map[c] = map.get(c) + 1
            else :
                map[c] = 1
        # Exploring all characters in String t and decreasing its frequency
        for c in t :
            if c not in map or map[c] == 0 :
                return False
            else :
                map[c] = map.get(c) - 1
        return True
```

→ PND
 2n2

→ T.C
 ↓
 O(n)
 n + n
 → S.C
 O(n)

58. Length of Last Word

Easy Topics Companies

Given a string `s` consisting of words and spaces, return the length of the last word in the string.

A **word** is a maximal **substring** consisting of non-space characters only.

Example 1:

Input: `s = "Hello World"`

Output: 5

"moon _ _ _ _ _ _ _ _ _ _"
 "hello _ _ _ _ _"
 ↑

Explanation: The last word is "world" with length 5.

Example 2:

Input: s = " fly me to the moon "
Output: 4

class Solution:

```
def lengthOfLastWord(self, s: str) -> int:
    count = 0
    # Ignore the trailing spaces from right
    i = len(s) - 1
    while i >= 0 and s[i] == ' ':
        i -= 1
    while i > -1 and s[i] != ' ':
        count += 1
        i -= 1
    return count
```

→ P.N. (58)

→ T.C → O(n)

→ S.C → O(1)

10⁵ a-----

10⁵ → 10⁵ → 6

165. Compare Version Numbers

Solved ✓

Medium

Topics

Companies

Hint

Given two **version strings**, `version1` and `version2`, compare them. A version string consists of **revisions** separated by dots `'.'`. The **value of the revision** is its **integer conversion** ignoring leading zeros.

To compare version strings, compare their revision values in **left-to-right order**. If one of the version strings has **fewer revisions**, treat the **missing revision values** as `0`.

Return the following:

- If `version1 < version2`, return `-1`.
- If `version1 > version2`, return `1`.
- Otherwise, return `0`.

"123.45.72"

"123.45.27.1"

"1.1.1" "1.1.001"

↓
0

"123.20.35" "123.20.35.0.0.0"

123

1 2 3
* * *

num $\begin{bmatrix} x \\ \checkmark \\ 0 \end{bmatrix}$

1234 . 25 . 0 . 0 . 0

1234 . 25

```
class Solution:
    def compareVersion(self, v1: str, v2: str) -> int:
        i, j = 0, 0
        while i < len(v1) or j < len(v2):
            # Find equivalent revision in int for version1
            num1 = 0
            while i < len(v1) and v1[i] != '.':
                num1 = (num1 * 10) + int(v1[i])
                i += 1
            # Find equivalent revision in int for version2
            num2 = 0
            while j < len(v2) and v2[j] != '.':
                num2 = (num2 * 10) + int(v2[j])
                j += 1
            if num1 < num2:
                return -1
            elif num1 > num2:
                return 1
            i += 1
            j += 1
        return 0
```

→ P.No : (165)

→ T.C → $O(m+n)$

→ S.C → $O(1)$

m → length of v1

n → length of v2